WORKFLOW

03

# Page Review PDF Workflow

Client Review Process Documentation

| PREPARED FOR | PREPARED BY | DATE | DOCUMENT |
| --- | --- | --- | --- |
| Precision Structures Inc. | EBYG Automation — SSC Digital Services | March 19, 2026 | 03 of 17 |

**Project:** Precision Structures Inc. Website **Prepared by:** SSC Development Team **Date:** March 18, 2026

## Overview

This document describes a repeatable workflow for generating PDF snapshots of every page on the Precision Structures website. These PDFs are delivered to the client for visual review and markup before changes go live.

## Pages Covered

| PAGE | FILE | STAGING URL PATH |
|---|---|---|
| Home | index.html | / |
| About | about.html | /about |
| Services | services.html | /services |
| Gallery | gallery.html | /gallery |
| Resources | resources.html | /resources |
| Contact | contact.html | /contact |

## Prerequisites

### Dependencies

Install the required packages in the project root:

```
npm install --save-dev @playwright/test
npx playwright install chromium
```

`@playwright/test` provides the browser automation engine. The second command downloads the Chromium binary that Playwright uses to render pages.

### Local Server or Staging URL

The script needs a running web server to capture pages. Two options:

1. **Local server** (for pre-deploy review):

   ```
   npx serve . -l 3000
   ```

   Base URL: `http://localhost:3000`

2. **Azure staging URL** (for post-deploy review): Base URL: the staging slot URL provided by Azure Static Web Apps (typically something like `https://<app-name>-develop.azurestaticapps.net`)

---

# Script Location

```
scripts/generate-review-pdfs.js
```

Run with:

```
node scripts/generate-review-pdfs.js
```

Optional flags:

| FLAG | DEFAULT | DESCRIPTION |
|---|---|---|
| `--base-url` | `http://localhost:3000` | Base URL to capture from |
| `--mobile` | `false` | Also generate mobile-viewport PDFs |
| `--output-dir` | `./review-pdfs` | Directory for generated files |

Example:

```
node scripts/generate-review-pdfs.js --base-url https://staging.example.com --mobile
```

---

# Script Behavior

## 1. Setup

- Launch headless Chromium via Playwright.
- Create the output directory (`/review-pdfs/`) if it does not exist.
- Record the current date and time for watermarking.

For each page in the list above:

1. Navigate to `{base-url}/{path}`.
2. Wait for `networkidle` (all assets loaded, no pending requests for 500ms).
3. Set viewport to **1440 × 900** (desktop).
4. Generate a PDF with the following settings:
   - **Format:** Letter (8.5 × 11 in)
   - **Print background:** enabled (captures background colors and images)
   - **Scale:** 0.75 (fits wide desktop layouts onto a printed page)
   - **Margins:** top 1in, bottom 1in, left 0.5in, right 0.5in
   - **Header:** page title and URL, right-aligned date
   - **Footer:** centered text reading `Review Copy - Precision Structures Inc.` with page number
5. Save as `review-pdfs/{page-name}-desktop-{YYYY-MM-DD}.pdf`.
6. If `--mobile` is set, resize viewport to **375 × 812** (iPhone-equivalent), repeat capture, save as `review-pdfs/{page-name}-mobile-{YYYY-MM-DD}.pdf`.

## 3. Header and Footer Template

Playwright's `page.pdf()` method accepts `headerTemplate` and `footerTemplate` as HTML strings.

**Header template:**

```
<div style="font-size:9px; width:100%; padding:0 40px; display:flex; justify-content:space-between;">
  <span class="title"></span>
  <span>URL: <span class="url"></span></span>
  <span>Generated: <span class="date"></span></span>
</div>
```

**Footer template:**

```
<div style="font-size:9px; width:100%; text-align:center; padding:0 40px;">
  Review Copy &ndash; Precision Structures Inc.  |  Page <span class="pageNumber"></span> of <spa
</div>
```

## 4. Index File

After all PDFs are generated, the script writes `review-pdfs/index.md` listing every file:

```
# Review PDFs - Precision Structures Inc.
Generated: 2026-03-18

## Desktop (1440x900)
- [Home](index-desktop-2026-03-18.pdf)
- [About](about-desktop-2026-03-18.pdf)
- [Services](services-desktop-2026-03-18.pdf)
- [Gallery](gallery-desktop-2026-03-18.pdf)
- [Resources](resources-desktop-2026-03-18.pdf)
- [Contact](contact-desktop-2026-03-18.pdf)

## Mobile (375x812)
- [Home](index-mobile-2026-03-18.pdf)
- ...
```

## Script Outline (Pseudocode)

```
const { chromium } = require("playwright");
const fs = require("fs");
const path = require("path");

const PAGES = [
  { name: "Home",      file: "index",     path: "/" },
  { name: "About",     file: "about",     path: "/about" },
  { name: "Services",  file: "services",  path: "/services" },
  { name: "Gallery",   file: "gallery",   path: "/gallery" },
  { name: "Resources", file: "resources", path: "/resources" },
  { name: "Contact",   file: "contact",   path: "/contact" },
];

const BASE_URL = process.argv.includes("--base-url")
  ? process.argv[process.argv.indexOf("--base-url") + 1]
  : "http://localhost:3000";

const INCLUDE_MOBILE = process.argv.includes("--mobile");
const OUTPUT_DIR = "./review-pdfs";
const DATE = new Date().toISOString().split("T")[0];

async function main() {
  fs.mkdirSync(OUTPUT_DIR, { recursive: true });

  const browser = await chromium.launch();
  const context = await browser.newContext();
  const indexLines = [`# Review PDFs - Precision Structures Inc.`, `Generated: ${DATE}`, "", "## Desktop (

  for (const page of PAGES) {
    const tab = await context.newPage();
    await tab.setViewportSize({ width: 1440, height: 900 });
    await tab.goto(`${BASE_URL}${page.path}`, { waitUntil: "networkidle" });

    const filename = `${page.file}-desktop-${DATE}.pdf`;
    await tab.pdf({
      path: path.join(OUTPUT_DIR, filename),
      format: "Letter",
      printBackground: true,
      scale: 0.75,
      margin: { top: "1in", bottom: "1in", left: "0.5in", right: "0.5in" },
      displayHeaderFooter: true,
      headerTemplate: `<div style="font-size:9px;width:100%;padding:0 40px;display:flex;justify-content:spa
      footerTemplate: `<div style="font-size:9px;width:100%;text-align:center;">Review Copy &ndash; Precis
    });
    indexLines.push(`- [${page.name}](${filename})`);

    if (INCLUDE_MOBILE) {
      await tab.setViewportSize({ width: 375, height: 812 });
      await tab.goto(`${BASE_URL}${page.path}`, { waitUntil: "networkidle" });
      const mobileFilename = `${page.file}-mobile-${DATE}.pdf`;
      await tab.pdf({ /* same options, adjusted scale */ });
      // add to mobile section of index
    }

    await tab.close();
```

```
  fs.writeFileSync(path.join(OUTPUT_DIR, "index.md"), indexLines.join("\n"));
  await browser.close();
  console.log(`Done. ${PAGES.length} page(s) captured to ${OUTPUT_DIR}/`);
}

main();
```

---

## Output Directory Structure

```
review-pdfs/
  index.md
  index-desktop-2026-03-18.pdf
  about-desktop-2026-03-18.pdf
  services-desktop-2026-03-18.pdf
  gallery-desktop-2026-03-18.pdf
  resources-desktop-2026-03-18.pdf
  contact-desktop-2026-03-18.pdf
  index-mobile-2026-03-18.pdf      (if --mobile)
  about-mobile-2026-03-18.pdf      (if --mobile)
  ...
```

Add `review-pdfs/` to `.gitignore` so generated files are not committed to the repository.

---

## Client Review and Markup Process

### How to Review the PDFs

Once the PDFs are generated, they are delivered to the client via email or a shared link (OneDrive, Google Drive, etc.). The client reviews each page and marks up any requested changes.

### Markup Methods

#### Option 1: Print and Annotate by Hand

1. Print the PDF.
2. Use a pen or marker to circle, underline, or annotate areas that need changes.
3. Write notes in the margins describing the desired change.
4. Scan or photograph the annotated pages and send them back.

#### Option 2: Digital Markup with Adobe Acrobat Reader (Free)

1. Open the PDF in Adobe Acrobat Reader (free download from get.adobe.com/reader).
2. Use the **Comment** tools in the toolbar:

- **Sticky Note** to add a comment at a specific location.
- **Drawing** tools to circle or arrow to specific elements.
- **Text Box** to type a note directly on the page.

3. Save the annotated PDF and email it back.

**Option 3: macOS Preview Markup**

1. Open the PDF in Preview (default on Mac).
2. Click the **Markup Toolbar** button (pen-tip icon).
3. Use shapes, text, arrows, and the sketch tool to annotate.
4. Save and return the file.

**Option 4: Microsoft Edge Built-In Markup**

1. Open the PDF in Microsoft Edge.
2. Use the **Draw** and **Highlight** tools in the PDF toolbar.
3. Save a copy and return it.

## What to Include in Markup

For each change, the client should note:

- **What** needs to change (e.g., "Update this phone number").
- **Where** on the page (circle or arrow to the element).
- **New content** if applicable (e.g., "Replace with: (801) 985-3000").
- **Priority** if some changes are more urgent than others.

## Turnaround

After receiving marked-up PDFs, the development team will:

1. Implement changes on the `develop` branch.
2. Deploy to the staging environment.
3. Generate a new set of review PDFs if needed.
4. Notify the client when ready for re-review.

# Troubleshooting

| ISSUE | SOLUTION |
|---|---|
| Script fails with "browser not found" | Run `npx playwright install chromium` |
| Pages render without styles | Ensure the local server is running and serving CSS/JS correctly |
| PDFs are blank or cut off | Check viewport dimensions; increase `waitUntil` timeout |
| Footer text not appearing | Ensure `displayHeaderFooter: true` and margins are large enough |
| Gallery images missing | Images may lazy-load; add a scroll-to-bottom step before capture |